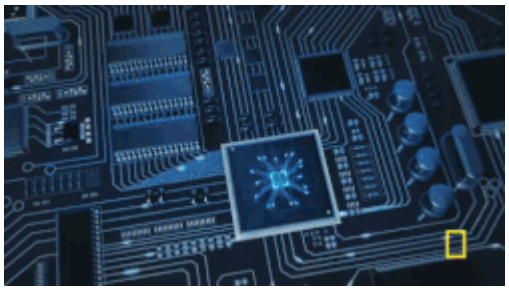
Minimierung Boolescher Funktionen

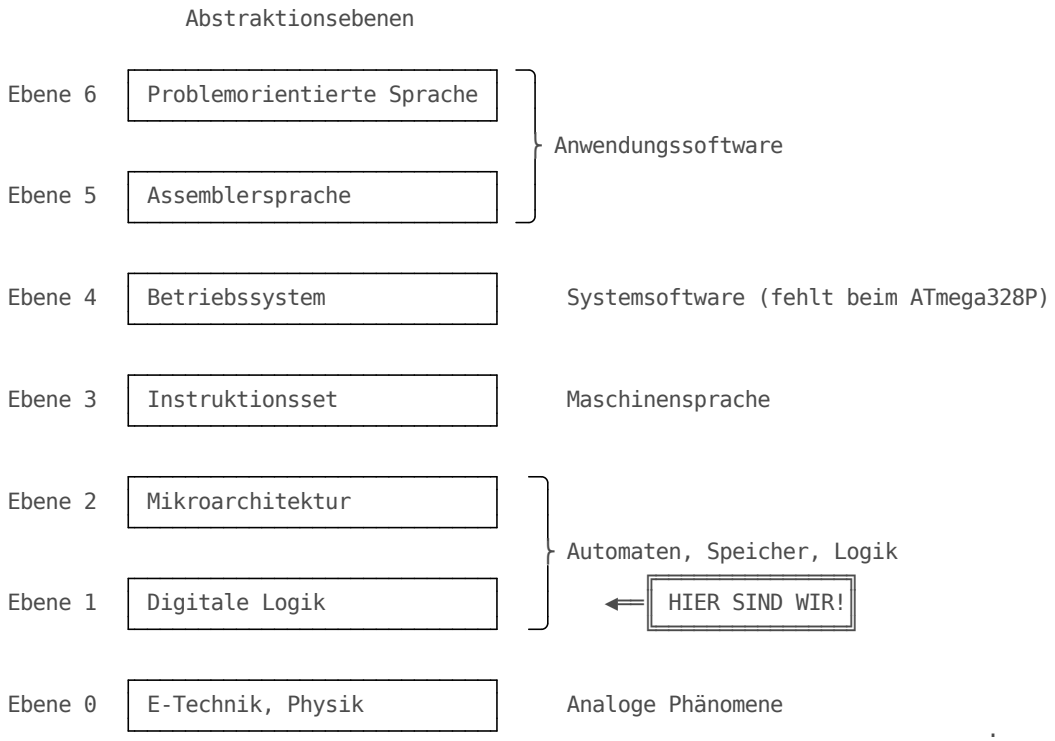
Parameter	Kursinformationen
Veranstaltung:	`Digitale Systeme / Eingebettete Systeme`
Semester:	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Verfahren zur Minimierung Boolescher Funktionen, Karnaugh-Veitch-Diagramme`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/03_Minimierung.md
Autoren:	Sebastian Zug & André Dietrich & Fabian Bär & Claude Code (Opus 5.5)



Lernziele: Nach dieser Vorlesung können Sie

- die Normalformen DNF und KNF sowie ihre kanonischen Formen KDNF und KKNF unterscheiden,
- aus einer Wahrheitstafel die KDNF und die KKNF einer Schaltfunktion aufstellen,
- Schaltfunktionen mit bis zu vier Variablen im Karnaugh-Veitch-Diagramm minimieren – über die Einsen zur DNF und über die Nullen zur KNF,
- Don't-Care-Einträge nutzen, um größere Schleifen und damit einfachere Ausdrücke zu bilden.

🗺️ **Diese Vorlesung in der Kurslandkarte →**



Rückblick: Hausaufgabe aus Vorlesung 02

Beginnen wir mit Teil B der Hausaufgabe, dem Datenblatt des UND-Gatters. Die zweite Frage führt direkt zum heutigen Thema, denn die Kreuze in der Function Table meinen das, was wir heute Don't Care nennen.

> **Aufgabe 3 aus Vorlesung 02:** Lesen Sie das Datenblatt des UND-Gatters SN74LVC2G08 von Texas Instruments.

a) Wie groß ist die maximale Verzögerung (*propagation delay* t_{pd}), mit der der Ausgang dem Eingang folgt, bei einer Versorgungsspannung von 3,3 V?

Bei $V_{CC} = 3{,}3\mathrm{V} \pm 0{,}3\mathrm{V}$ ist t_{pd} höchstens 5,7 ns. Das klingt wenig, summiert sich aber: Zehn solcher Gatter hintereinander können bis zu 57 ns brauchen. Kürzere Wege durch eine Schaltung machen sie also auch schneller.

b) Was bedeuten die Kreuze in der Wahrheitstafel (*Function table*)?

X steht für einen beliebigen Pegel. Liegt ein Eingang auf L, ist der andere egal, der Ausgang bleibt L. Dort sagt das X: Der **Eingang** spielt keine Rolle. Heute nutzen wir denselben Gedanken am **Ausgang**: Für manche Eingaben ist egal, was herauskommt.

c) Können Sie mit dem Gatter auch eine Negation des Eingangssignals realisieren? Begründen Sie mit der Wahrheitstafel.

Nicht möglich. Laut Tabelle ist der Ausgang nur bei A = B = H auf H. Verbinden Sie beide Eingänge oder legen Sie einen fest auf H, folgt der Ausgang dem Eingang: $Y = A$. Der Ausgang geht also nie auf H, weil ein Eingang auf L geht. Dafür braucht es ein NICHT oder ein NAND.

Ausgangspunkt

In der letzten Vorlesung haben Sie Schaltfunktionen durch algebraische Umformungen vereinfacht. Das gelingt, aber Sie müssen die richtigen Gesetze in der richtigen Reihenfolge finden, und am Ende wissen Sie nicht, ob es noch kürzer geht.

Rückblick: Algebraische Minimierung (Vorlesung 02)

$$\begin{aligned} f(x_1, x_2, x_3) &= x_1 \cdot x_2 \cdot \bar{x}_3 + x_1 \cdot x_2 \cdot x_3 + x_1 \cdot \bar{x}_2 \cdot x_3 \\ &\dots \text{ mehrere Umformungsschritte später } \dots \\ &= x_1 \cdot x_2 + x_1 \cdot x_3 \end{aligned}$$

Die algebraische Methode hat drei Schwächen: Es gibt kein festes Vorgehen, viele Schritte bedeuten viele Fehlerquellen, und das Ergebnis verrät nicht, ob es minimal ist.

Das Ziel bleibt dasselbe wie in der letzten Vorlesung. Heute ändert sich der Weg: Wir ordnen die Wahrheitstafel so an, dass man zusammenfassbare Terme sehen kann. Das funktioniert bis zu vier Variablen. Für mehr Variablen kommt in der nächsten Vorlesung ein Algorithmus.

> **Das Ziel bleibt gleich:** Weniger Gatter bedeuten geringere Kosten, kürzere Laufzeiten und weniger Energiebedarf.
>
> **Der Weg ändert sich:** Statt nach passenden Gesetzen zu suchen, gehen wir nach einem festen Verfahren vor.

Heute: Normalformen als gemeinsamer Startpunkt und das Karnaugh-Veitch-Diagramm für bis zu vier Variablen. **Nächste Vorlesung:** das Verfahren von Quine-McCluskey für beliebig viele Variablen.

Herausforderung

Beginnen wir mit einem konkreten Beispiel. Die beiden Ausdrücke sehen verschieden aus. Beschreiben sie dieselbe Funktion?

> **Die Aufgabe:** Zeigen Sie, dass diese beiden Funktionen äquivalent sind!

$$\begin{aligned} f(x_1, x_2, x_3, x_4) &= x_3\bar{x}_1 + x_4\bar{x}_3\bar{x}_2 + \bar{x}_4 x_3 x_2 \bar{x}_1 + x_4 x_3 \bar{x}_2 x_1 \\ g(x_1, x_2, x_3, x_4) &= x_3\bar{x}_1 + x_4\bar{x}_2 \end{aligned}$$

Versuchen wir es zunächst mit algebraischen Umformungen wie in der letzten Vorlesung. Nach dem Absorptionsgesetz geht es scheinbar nicht weiter. Der Beweis gelingt zwar, aber nur mit einem Schritt, auf den man erst einmal kommen muss.

Versuch 1: Algebraische Vereinfachung

$$\begin{aligned} f(x_1, x_2, x_3, x_4) &= x_3\bar{x}_1 + x_4\bar{x}_3\bar{x}_2 + \bar{x}_4 x_3 x_2\bar{x}_1 + x_4 x_3\bar{x}_2 x_1 \\ &= x_3\bar{x}_1 + \bar{x}_4 x_3 x_2\bar{x}_1 + x_4 x_3\bar{x}_2 x_1 + x_4\bar{x}_3\bar{x}_2 && (\text{Kommutativgesetz}) \\ &= x_3\bar{x}_1 + x_4 x_3\bar{x}_2 x_1 + x_4\bar{x}_3\bar{x}_2 && (\text{Absorptionsgesetz}) \\ &= \ldots \text{ und jetzt?} \end{aligned}$$

> Der Beweis ist möglich, aber der nächste Schritt ist nicht offensichtlich: Das Distributivgesetz vereinfacht $x_3\bar{x}_1 + x_4 x_3\bar{x}_2 x_1$ zu $x_3\bar{x}_1 + x_4 x_3\bar{x}_2$. Erst dann lässt sich x_3 gegen den letzten Term zusammenfassen. Nichts am Ausdruck verrät, dass man genau hier ansetzen muss.

Alternativ prüfen wir die Äquivalenz über die Wahrheitstafel. Das ist mühsam, aber systematisch: Die Spalten f und g stimmen in jeder Zeile überein. Die rot markierten Zeilen deckt der kurze Term $x_4\bar{x}_2$ ab.

Versuch 2: Nachweis über die Wahrheitstafel

x_4	x_3	x_2	x_1	f	g	
0	0	0	0	0	0	
0	0	0	1	0	0	
0	0	1	0	0	0	
0	0	1	1	0	0	
0	1	0	0	1	1	$x_3\bar{x}_1$
0	1	0	1	0	0	
0	1	1	0	1	1	$\bar{x}_4x_3x_2\bar{x}_1$, $x_3\bar{x}_1$
0	1	1	1	0	0	
1	0	0	0	1	1	$x_4\bar{x}_3\bar{x}_2$
1	0	0	1	1	1	$x_4\bar{x}_3\bar{x}_2$
1	0	1	0	0	0	
1	0	1	1	0	0	
1	1	0	0	1	1	$x_3\bar{x}_1$
1	1	0	1	1	1	$x_4x_3\bar{x}_2x_1$
1	1	1	0	1	1	$x_3\bar{x}_1$
1	1	1	1	0	0	

Die Spalten f und g stimmen in allen 16 Zeilen überein, damit ist die Äquivalenz gezeigt. Die Tafel verdoppelt sich aber mit jeder Variablen: 16 Zeilen bei vier, 32 bei fünf, 64 bei sechs Variablen.

Was ist damit gewonnen? Rechnet man alles in Gatter mit zwei Eingängen um, schrumpft die Schaltung von zwölf auf drei Gatter. Negationen sind dabei nicht mitgezählt.

Was die Minimierung bringt (umgerechnet in Gatter mit zwei Eingängen, ohne Negationen):

Ausdruck	Terme mit Variablen	UND	ODER
f	2, 3, 4 und 4	9	3
g	2 und 2	2	1

Beginnen wir mit dem, was jedes Verfahren braucht: einer einheitlichen Schreibweise, der Normalform.

Aufstellen der Normalform

Eine Wahrheitstafel beschreibt eine Funktion eindeutig. Ein Ausdruck tut das nicht, denn dieselbe Funktion lässt sich beliebig verschieden aufschreiben. Die kanonischen Normalformen stellen diese Eindeutigkeit wieder her.

Format	Bedeutung
Wahrheitstafel	eindeutig
Boolescher Ausdruck	uneindeutig
kanonische Normalform (KDNF, KKNF)	eindeutig
DNF oder KNF ohne kanonische Form	uneindeutig
Schaltnetz	uneindeutig

Vier Begriffe brauchen wir: Produktterm und Summenterm, und als deren vollständige Sonderfälle Minterm und Maxterm. Vollständig heißt, dass jede Variable genau einmal vorkommt.

Für die Darstellung der Normalform benötigen wir zunächst weitere Begriffsdefinitionen.

Produktterm

- Konjunktion (UND-Verknüpfung) von Variablen (ggf. negiert)
- Beispiele: $x \cdot y$, $x \cdot \overline{y}$, $x \cdot y \cdot \overline{z}$

Summenterm

- Disjunktion (ODER-Verknüpfung) von Variablen (ggf. negiert)
- Beispiele: $x + y$, $x + \overline{y}$, $x + y + z$

Minterm

- Produktterm, in dem jede Variable einer booleschen Funktion genau einmal vorkommt (einfach oder negiert)
- Beispiel: $x \cdot y \cdot \overline{z}$ ist ein Minterm der Funktion $f(x, y, z)$

Maxterm

- Summenterm, in dem jede Variable einer booleschen Funktion genau einmal vorkommt (einfach oder negiert)
- Beispiel: $x + y + z$ ist ein Maxterm der Funktion $f(x, y, z)$

Aus diesen Termen setzen sich die Normalformen zusammen. Bei der DNF werden Produktterme mit ODER verknüpft, bei der KNF Summenterme mit UND. Die kanonischen Formen verwenden ausschließlich Minterme beziehungsweise Maxterme.

Und nun in der Kombination ….

Disjunktive Normalform (DNF, Summe von Produkttermen)

- Disjunktion von Produkttermen
- Beispiel: $(x \cdot y) + (x \cdot y \cdot z)$

Konjunktive Normalform (KNF, Produkt von Summentermen)

- Konjunktion von Summentermen
- Beispiel: $w \cdot (x + y) \cdot (x + y + z)$

Kanonische Disjunktive Normalform (KDNF)

- eindeutige Darstellung einer booleschen Funktion f als Disjunktion von Mintermen
- Beispiel: $(\overline{x} \cdot y \cdot z) + (x \cdot \overline{y} \cdot \overline{z}) + (\overline{x} \cdot y \cdot \overline{z})$ ist KDNF von $f(x, y, z)$

Kanonische Konjunktive Normalform (KKNF)

- eindeutige Darstellung einer booleschen Funktion f als Konjunktion von Maxtermen
- Beispiel: $(x + \overline{y}) \cdot (\overline{x} + y) \cdot (\overline{x} + \overline{y})$ ist KKNF von $f(x, y)$

Die KDNF lesen Sie direkt aus der Wahrheitstafel ab: Jede Zeile mit einer Eins liefert einen Minterm.

> „KDNF = Disjunktion von Mintermen“

Jede Boolesche Funktion lässt sich als genau eine KDNF darstellen!

Bildung der KDNF für n -stellige Boolesche Funktion f:

- für jede Zeile der Wahrheitstabelle mit $f(x_1, x_2, ..., x_n) = 1$ wird ein Minterm aufgestellt
- hierin wird jede Variable x_i negiert, wenn in der entsprechenden Zeile der Wert der Variablen 0 ist

Am Beispiel der Aufgabe vom Beginn der Vorlesung ergibt sich daraus:

x_4	x_3	x_2	x_1	f	**Minterme**
0	0	0	0	0	
0	0	0	1	0	
0	0	1	0	0	
0	0	1	1	0	
0	1	0	0	1	$\overline{x}_4 x_3 \overline{x}_2 \overline{x}_1$
0	1	0	1	0	
0	1	1	0	1	$\overline{x}_4 x_3 x_2 \overline{x}_1$
0	1	1	1	0	
1	0	0	0	1	$x_4 \overline{x}_3 \overline{x}_2 \overline{x}_1$
1	0	0	1	1	$x_4 \overline{x}_3 \overline{x}_2 x_1$
1	0	1	0	0	
1	0	1	1	0	
1	1	0	0	1	$x_4 x_3 \overline{x}_2 \overline{x}_1$
1	1	0	1	1	$x_4 x_3 \overline{x}_2 x_1$
1	1	1	0	1	$x_4 x_3 x_2 \overline{x}_1$
1	1	1	1	0	

Damit ergibt sich die Kanonische Disjunktive Normalform obiger Wahrheitstafel zu

$$\begin{aligned} f(x_1, x_2, x_3, x_4) = \quad & \overline{x}_4 x_3 \overline{x}_2 \overline{x}_1 + \\ & \overline{x}_4 x_3 x_2 \overline{x}_1 + \\ & x_4 \overline{x}_3 \overline{x}_2 \overline{x}_1 + \\ & x_4 \overline{x}_3 \overline{x}_2 x_1 + \\ & x_4 x_3 \overline{x}_2 \overline{x}_1 + \\ & x_4 x_3 \overline{x}_2 x_1 + \\ & x_4 x_3 x_2 \overline{x}_1 \end{aligned}$$

Vergleichen Sie die Darstellung mit dem Ausdruck f aus der Aufgabe am Anfang!

> Die Darstellung als KDNF ist abgesehen von Reihenfolge eindeutig!

Für die KKNF gehen Sie genauso vor, nur über die Nullen: Jede Zeile mit einer Null liefert einen Maxterm, und die Negation dreht sich um.

> „KKNF = Konjunktion von Maxtermen“

> Jede Boolesche Funktion lässt sich als genau eine KKNF darstellen!

Bildung der KKNF für n -stellige Boolesche Funktion f:

- für jede Zeile der Wahrheitstabelle mit $f(x_1, x_2, ..., x_n) = 0$ wird ein Maxterm aufgestellt
- hierin wird jede Variable x_i negiert, wenn in der entsprechenden Zeile der Wert der Variablen 1 ist

Am Beispiel der Aufgabe vom Beginn der Vorlesung ergibt sich daraus:

x_4	x_3	x_2	x_1	f
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Damit ergibt sich die Kanonische Konjunktive Normalform obiger Wahrheitstafel zu

$$
\begin{aligned}
f(x_1, x_2, x_3, x_4) = \quad & (x_4 + x_3 + x_2 + x_1) \cdot \\
& (x_4 + x_3 + x_2 + \bar{x}_1) \cdot \\
& (x_4 + x_3 + \bar{x}_2 + x_1) \cdot \\
& (x_4 + x_3 + \bar{x}_2 + \bar{x}_1) \cdot \\
& (x_4 + \bar{x}_3 + x_2 + \bar{x}_1) \cdot \\
& (x_4 + \bar{x}_3 + \bar{x}_2 + \bar{x}_1) \cdot \\
& (\bar{x}_4 + x_3 + \bar{x}_2 + x_1) \cdot \\
& (\bar{x}_4 + x_3 + \bar{x}_2 + \bar{x}_1) \cdot \\
& (\bar{x}_4 + \bar{x}_3 + \bar{x}_2 + \bar{x}_1)
\end{aligned}
$$

Die Darstellung als KKNF ist abgesehen von Reihenfolge eindeutig!

Welche der beiden Formen kürzer ist, hängt davon ab, ob die Wahrheitstafel mehr Einsen oder mehr Nullen enthält. Prüfen Sie danach mit den beiden Fragen, ob Sie die Begriffe sicher unterscheiden.

Satz: Jede KDNF kann in eine KKNF umgewandelt, jede KKNF kann in eine KDNF umgewandelt werden.

Zwei Darstellungen Boolescher Funktionen sind äquivalent, wenn sie durch Umformungen gemäß den Sätzen der Booleschen Algebra auf die gleiche KDNF bzw. KKNF zurückgeführt werden können

Welche Form erweist sich in welcher Situation als günstiger?

Eine KDNF ist günstiger als eine KKNF, wenn nur für wenige Kombinationen der Eingabewerte $f(x_1, x_2, ..., x_n) = 1$ gilt. Umgekehrt, wenn nur für wenige Kombinationen der Eingabewerte $f(x_1, x_2, ..., x_n) = 0$ gilt, ist die KKNF vorzuziehen.

Für die OR Funktion bedeutet das (wenig überraschend):

x	y	f
0	0	0
0	1	1
1	0	1
1	1	1

KDNF: $f = \bar{x} \cdot y + x \cdot \bar{y} + x \cdot y$

KKNF: $f = x + y$

Eine Funktion von vier Variablen ist in 6 Zeilen der Wahrheitstafel gleich 1. Wie viele Maxterme hat ihre KKNF?

Welcher Ausdruck ist eine KDNF einer Funktion $f(x, y, z)$?

- ◯ $xy + \bar{x}z$
- ◯ $xyz + \bar{x}y\bar{z}$
- ◯ $(x + y + z)(\bar{x} + y + z)$
- ◯ $x + y + z$

Minimierung von Schaltfunktionen mit dem KV-Diagramm

Minimal kann Verschiedenes heißen, deshalb legen wir es fest: zuerst möglichst wenige Terme, dann möglichst wenige Literale. Das Grundprinzip aller Verfahren ist dasselbe: Zwei Terme, die sich nur in einer Variablen unterscheiden, lassen sich zu einem kürzeren zusammenfassen.

- Wann heißt eine Darstellung minimal ?
  - kleinste Anzahl an Gattern ?
  - kleinste Anzahl an Verbindungsleitungen ?
  - kleinste Anzahl an Produkttermen bzw. Summentermen ?

> **Minimal heißt in dieser Vorlesung:** unter allen DNF (oder KNF) einer Funktion möglichst wenige Terme, bei Gleichstand möglichst wenige Literale. Ein **Literal** ist ein Vorkommen einer Variablen, negiert oder nicht: $x\bar{y} + xz$ hat 2 Terme und 4 Literale, aber nur 3 Variablen. Es kann mehrere gleich gute Minimalformen geben.

Lösungsmöglichkeiten:

1. geeignetes Umformen mit Gesetzen der Booleschen Algebra
2. Einsatz eines graphischen Verfahrens (z.B. ein Karnaugh-Veitch-Diagramm), nur möglich bei Schaltfunktionen mit wenigen Variablen
3. algorithmisches Minimieren (z.B. Verfahren nach Quine-McCluskey, nächste Vorlesung), geeignet auch für Schaltfunktionen mit vielen Variablen

Ausgangspunkt für die Minimierung von Booleschen Ausdrücken ist das neutrale Element einer Operation. Für eine `ODER` Verknüpfung bewirkt eine falsche Aussage und für eine `UND` Operation eine wahre Aussage keine Veränderung des Gesamtausdruckes.

Dafür nutzen wir die **Resolutionsregel**:

Disjunktive Normalform	**Konjunktive Normalform**
$\begin{aligned} f(x,y) &= x \cdot y + x \cdot \overline{y} \\ &= x \cdot (y + \overline{y}) \\ &= x \cdot (1) \end{aligned}$	$\begin{aligned} f(x,y) &= (x + y) \cdot (x + \overline{y}) \\ &= x + (y \cdot \overline{y}) \\ &= x + (0) \end{aligned}$
Wenn sich in einer disjunktiven Normalform zwei Produktterme nur in genau einer komplementären Variablen unterscheiden, so können beide durch ihren gemeinsamen Teil ersetzt werden!	Wenn sich in einer konjunktiven Normalform zwei Summenterme nur in genau einer komplementären Variablen unterscheiden, so können beide durch ihren gemeinsamen Teil ersetzt werden!

Karnaugh-Veitch-Diagramme - Konzept

Die Idee: Wir schreiben die Wahrheitstafel als Rechteck. Dann liegen Terme, die sich zusammenfassen lassen, sichtbar nebeneinander.

Jede zweistellige Boolesche Funktion lässt sich mit den Kombinationen ihrer Variablen in einer tabellarischen Darstellung visualisieren:

$A = \overline{x} \cdot \overline{y} + \overline{x} \cdot y + x \cdot \overline{y} + x \cdot y$

	$\overline{x}$	x
$\overline{y}$	$\overline{x} \cdot \overline{y}$	$x \cdot \overline{y}$
y	$\overline{x} \cdot y$	$x \cdot y$

Mit dieser umgeordneten Wahrheitstafel lässt sich eine Boolesche Funktion auf einen Blick erfassen.

Beispiel 1: $f = x \cdot \overline{y} + x \cdot y$

	$\overline{x}$	x
$\overline{y}$	0	1
y	0	1

Beispiel 2: $f = x \cdot \overline{y} + \overline{x} \cdot y$

	$\overline{x}$	x
$\overline{y}$	0	1
y	1	0

Damit Nachbarschaft im Bild auch Nachbarschaft in der Logik bedeutet, folgen die Spalten und Zeilen nicht der Zählreihenfolge, sondern dem Gray-Code. Beachten Sie die Reihenfolge null null, null eins, eins eins, eins null. Auch die Ränder gehören zusammen, das Diagramm schließt sich wie ein Torus.

Dieses Konzept lässt sich auf mehr Variablen übertragen, von Hand ist es aber nur bis zu vier Variablen übersichtlich. Wichtig dabei ist, dass benachbarte Kombinationen von Variablen sich nur in **einer** Variable unterscheiden. (Siehe Gray-Code)

Der Gray-Code ist ein stetiger Code, bei dem sich benachbarte Codewörter nur in einer einzigen binären Ziffer unterscheiden.

Dezimalzahl	**Binärcode**	**Gray-Code**
0	000	000
1	001	001
2	010	011
3	011	010

	$\bar{x}\bar{y}$	$\bar{x}y$	xy	$x\bar{y}$
$\bar{z}$	$\bar{x} \cdot \bar{y} \cdot \bar{z}$	$\bar{x} \cdot y \cdot \bar{z}$	$x \cdot y \cdot \bar{z}$	$x \cdot \bar{y} \cdot \bar{z}$
z	$\bar{x} \cdot \bar{y} \cdot z$	$\bar{x} \cdot y \cdot z$	$x \cdot y \cdot z$	$x \cdot \bar{y} \cdot z$

> Jeweils nur ein Wechsel von einer Variable pro Zeilen-/Spaltenübergang! Das Diagramm schließt sich dabei wie ein Torus: Linker und rechter Rand sowie oberer und unterer Rand grenzen aneinander!

Nun das Verfahren in vier Schritten. Das rote Feldpaar unten unterscheidet sich nur in z, also fällt z heraus.

Vorgehen zur Minimierung der KDNF einer n-stelligen Funktion f:

1. Für jeden Minterm mit $f(x_1, ..., x_n) = 1$ wird eine 1 ins Diagramm eingetragen.
2. Für jeden Maxterm mit $f(x_1, ..., x_n) = 0$ wird eine 0 ins Diagramm eingetragen.
3. Möglichst wenige und möglichst große Rechtecke (Schleifen) aus Einsen werden markiert, wobei gegenüberliegende Ränder aneinandergrenzen und
   - jedes Rechteck aus 2^k Feldern (mit $0 \leq k \leq n$) besteht und
   - alle Einsen überdeckt werden müssen.
4. Jede Schleife wird nach der Resolutionsregel zu einem Produktterm: Variablen, die in der Schleife wechseln, fallen weg. Variablen, die konstant bleiben, stehen im Term, bei Wert 1 einfach, bei Wert 0 negiert. Die Produktterme werden summiert.

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	0	1	0	0
$\bar{y}z$	0	1	0	0
yz	0	0	0	0
$y\bar{z}$	0	0	0	0

Welcher Ausdruck steht hinter dem markierten Bereich?

$$\begin{aligned} f &= \bar{w}x\bar{y}\bar{z} + \bar{w}x\bar{y}z \\ &= \bar{w}x\bar{y}(\bar{z} + z) \\ &= \bar{w}x\bar{y} \cdot 1 \\ &= \bar{w}x\bar{y} \end{aligned}$$

Je größer die Schleife, desto kürzer der Term: Vier Felder lassen zwei Variablen herausfallen, zwei Felder nur eine.

Beispiel 1: $\bar{w}\bar{y}$

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	0
$\bar{y}z$	1	1	0	0
yz	0	0	1	0
$y\bar{z}$	0	0	1	0

Beispiel 2: wxy

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	0
$\bar{y}z$	1	1	0	0
yz	0	0	1	0
$y\bar{z}$	0	0	1	0

Eine zu kleine Schleife ist kein logischer Fehler, die Funktion stimmt weiterhin. Sie ist nur nicht minimal. Schleifen dürfen sich dafür überlappen, und über den Rand hinweg gehen sie ebenfalls.

Regeln zur Bildung der Schleifen:

- Fangen Sie mit isolierten Zellen an. Die entsprechenden Minterme können nicht mehr vereinfacht werden.
- Nehmen Sie dann Einsen, die nur in einer einzigen größtmöglichen Schleife liegen können. Diese Schleifen sind unverzichtbar.
- Bilden Sie für die restlichen Einsen die größtmöglichen Schleifen, zuerst für die Einsen mit den wenigsten gleichwertigen Nachbarzellen. Eine Schleife umfasst 2^k Zellen.
- Überlappungen führen nur dann zu kürzeren Ausdrücken, wenn dadurch größere Schleifen gebildet werden können.
- Die minimale Funktion besteht aus möglichst wenigen Schleifen, die jede für sich möglichst groß sind.

Vermeiden Sie also „Einzelelemente“:

Korrekt, aber nicht minimal - $f = \bar{w}\bar{y} + \bar{w}xyz$

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	0
$\bar{y}z$	1	1	0	0
yz	0	1	0	0
$y\bar{z}$	0	0	0	0

Minimal - $f = \bar{w}\bar{y} + \bar{w}xz$ (die doppelte 1 zeigt ein Feld, das in beiden Schleifen liegt)

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	0
$\bar{y}z$	1	1 1	0	0
yz	0	1	0	0
$y\bar{z}$	0	0	0	0

Schleifen dürfen über den Rand reichen, weil gegenüberliegende Ränder aneinandergrenzen.

Welche Felder dürfen im Diagramm mit vier Variablen zusammen eine Schleife bilden?

- [ ] das oberste und das unterste Feld derselben Spalte
- [ ] die vier Eckfelder
- [ ] drei nebeneinanderliegende Felder
- [ ] zwei Felder, die sich nur diagonal berühren
- [ ] vier Felder in L-Form

Beispiele

Decken Sie die Formel gedanklich ab und lesen Sie jeden Term selbst aus dem Diagramm: Welche Variablen bleiben in der Schleife konstant?

$f = \bar{w}x$

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{z}$	0	1	0	0
z	0	1	0	0

$f = z$

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{z}$	0	0	0	0
z	1	1	1	1

$f = x\bar{z}$

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	0	1	1	0
$\bar{y}z$	0	0	0	0
yz	0	0	0	0
$y\bar{z}$	0	1	1	0

$f = \bar{x}\bar{z}$

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	0	0	1
$\bar{y}z$	0	0	0	0
yz	0	0	0	0
$y\bar{z}$	1	0	0	1

Lesen Sie diese Funktion selbst ab. Die Felder stehen schon im Diagramm, Sie bilden die Schleifen. Das Diagramm meldet, ob eine Auswahl eine gültige Schleife ist und ob Ihr Ergebnis minimal ist.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1 0	0 4	0 12	1 8
$\bar{y}z$	0 1	1 5	1 13	1 9
yz	0 3	0 7	0 15	1 11
$y\bar{z}$	1 2	0 6	0 14	1 10

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

$f = ?$

Noch nicht überdeckt: 8 Einsen.

Nun die ganze Kette: Tragen Sie die Wahrheitstafel unter Felder eintragen ein, wechseln Sie zu Schleifen bilden und lesen Sie das Ergebnis ab.

Und jetzt Sie!

Tragen Sie die Wahrheitstafel unter „Felder eintragen“ ein, bilden Sie dann die Schleifen.

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)



Schleife übernehmen | Lösung zeigen | Zurücksetzen

Klick wechselt 0 → 1 → D.

$f = 0$

	$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
$\bar{C}$	1	1	0	1
C	1	0	0	0

$$\bar{A}\bar{B} + \bar{A}\bar{C} + \bar{B}\bar{C}$$

Optimierungskriterien

Schleifen dürfen sich überlappen, aber nicht jede Überlappung lohnt sich. Die drei Varianten überdecken alle dieselben Einsen, kosten aber unterschiedlich viele Operationen.

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	1
$\bar{y}z$	0	1	1	1
yz	0	1	1	0
$y\bar{z}$	0	0	0	0

Welche Möglichkeiten haben wir ausgehend von der ersten Gruppe?

Ein _ zeigt in den Tabellen, dass ein Feld zu einer zweiten Schleife gehört.

Variante 1: $xz + \bar{x}\bar{y}\bar{z} + \bar{w}x\bar{y} + w\bar{x}\bar{y}$ (10 Operationen)

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	1 _
$\bar{y}z$	0	1 _	1	1
yz	0	1	1	0
$y\bar{z}$	0	0	0	0

Variante 2: $xz + \bar{x}\bar{y}\bar{z} + \bar{w}x\bar{y} + w\bar{y}z$ (10 Operationen)

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	1
$\bar{y}z$	0	1	1	1
yz	0	1	1	0
$y\bar{z}$	0	0	0	0

Variante 3: $xz + \bar{w}\bar{y}\bar{z} + w\bar{x}\bar{y}$ (7 Operationen)

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	1	0	1
$\bar{y}z$	0	1	1	1
yz	0	1	1	0
$y\bar{z}$	0	0	0	0

Überlappung lohnt sich nur, wenn dadurch größere Schleifen entstehen und zum Beispiel eine isolierte 1 vermieden wird.

Bilden Sie Variante 1 selbst nach und vergleichen Sie die Rückmeldung mit Variante 3. Ein × am Term entfernt die Schleife wieder.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\overline{w}\,\overline{x}$	$\overline{w}x$	wx	$w\overline{x}$
$\overline{y}\,\overline{z}$	1 0	1 4	0 12	1 8
$\overline{y}z$	0 1	1 5	1 13	1 9
yz	0 3	1 7	1 15	0 11
$y\overline{z}$	0 2	0 6	0 14	0 10

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

$f = ?$

Noch nicht überdeckt: 8 Einsen.

Karnaugh-Veitch für Maxterme

Für die KNF zählen die Nullen. Sie bilden Schleifen um die Nullen und erhalten Summenterme, die mit UND verknüpft werden.

Das Karnaugh-Veitch-Diagramm lässt sich analog für Produkte von Summen aufstellen. Im Unterschied zur Minterm-Variante werden hier aber die Nullen erfasst - diese entsprechen ja auch den resultierenden Maxtermen.

Nehmen wir folgende Wahrheitstafel an:

w	x	y	z	A
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Die Funktion hat 8 Einsen und 8 Nullen, also 8 Minterme und 8 Maxterme.

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	0	0	1
$\bar{y}z$	0	1	1	1
yz	0	1	1	0
$y\bar{z}$	1	0	0	0

Am einfachsten lesen Sie die KNF über die Negation ab: Die Nullen von A sind die Einsen von $\bar{A}$.

- Nullen als DNF von $\bar{A}$ zusammenfassen: $\bar{A} = \bar{w}\bar{x}z + x\bar{z} + w\bar{x}y$
- De Morgan anwenden: $A_{KNF} = \bar{w}\bar{x}z + x\bar{z} + w\bar{x}y = (w + x + \bar{z})(\bar{x} + z)(\bar{w} + x + \bar{y})$

Direkt aus dem Diagramm heißt das: Jede Schleife liefert einen Summenterm aus den Variablen, die in ihr konstant bleiben, aber umgekehrt wie bei der DNF: Wert 1 erscheint negiert, Wert 0 einfach.

Prüfen Sie das hier: Mit „KNF (Nullen)“ bilden Sie die Schleifen um die Nullen, die Einsen dürfen dann nicht darin liegen.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1 (0)	0 (4)	0 (12)	1 (8)
$\bar{y}z$	0 (1)	1 (5)	1 (13)	1 (9)
yz	0 (3)	1 (7)	1 (15)	0 (11)
$y\bar{z}$	1 (2)	0 (6)	0 (14)	0 (10)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 8 Einsen.

Hier die Lösung als KNF. Jede der drei Schleifen um die Nullen liefert einen Summenterm. Die Variablen, die in der Schleife konstant bleiben, erscheinen darin umgekehrt: eine 1 negiert, eine 0 einfach.

Lösung als KNF: Die drei Schleifen um die Nullen in der Tabelle oben liefern je einen Summenterm.

Schleife	Felder	konstant in der Schleife	Summenterm
rot	Spalte $\bar{w}\bar{x}$, Zeilen $\bar{y}z$ und yz	$w = 0, x = 0,$ $z = 1$	$w + x + \bar{z}$
grün	Spalten $\bar{w}x$ und wx, Zeilen $\bar{y}\bar{z}$ und $y\bar{z}$ (über den Rand)	$x = 1, z = 0$	$\bar{x} + z$
blau	Spalte $w\bar{x}$, Zeilen yz und $y\bar{z}$	$w = 1, x = 0,$ $y = 1$	$\bar{w} + x + \bar{y}$

$$A_{KNF} = (w + x + \bar{z})(\bar{x} + z)(\bar{w} + x + \bar{y})$$

Liest man dasselbe Diagramm über die Einsen, entsteht eine DNF. Dass beide dieselbe Funktion beschreiben, zeigen wir durch Ausmultiplizieren. Dafür reichen die Gesetze aus der letzten Vorlesung.

Äquivalenz zur DNF: Über die Einsen liefert dasselbe Diagramm die DNF

$$A_{DNF} = \bar{w}\bar{x}\bar{z} + xz + w\bar{x}\bar{y}$$

Ausmultiplizieren der KNF führt genau auf diese DNF:

$$\begin{aligned} A_{KNF} &= (w + x + \bar{z})(\bar{x} + z)(\bar{w} + x + \bar{y}) \\ &= (w\bar{x} + wz + xz + \bar{x}\bar{z})(\bar{w} + x + \bar{y}) && (1) \\ &= (w\bar{x} + xz + \bar{x}\bar{z})(\bar{w} + x + \bar{y}) && (2) \\ &= w\bar{x}\bar{y} + \bar{w}xz + xz + xz\bar{y} + \bar{w}\bar{x}\bar{z} + \bar{x}\bar{y}\bar{z} && (3) \\ &= w\bar{x}\bar{y} + xz + \bar{w}\bar{x}\bar{z} + \bar{x}\bar{y}\bar{z} && (4) \\ &= \bar{w}\bar{x}\bar{z} + xz + w\bar{x}\bar{y} = A_{DNF} && (5) \end{aligned}$$

1. Die ersten beiden Klammern ausmultiplizieren, $x\bar{x} = 0$ und $z\bar{z} = 0$ fallen weg.
2. $wz = wz(x + \bar{x}) = wxz + w\bar{x}z$. Den ersten Teil absorbiert xz, den zweiten $w\bar{x}$, also fällt wz weg.
3. Mit der dritten Klammer ausmultiplizieren, $w\bar{w} = 0$, $x\bar{x} = 0$ und $xz \cdot x = xz$.
4. Absorption: xz enthält $\bar{w}xz$ und $xz\bar{y}$.
5. Wie in Schritt 2: $\bar{x}\bar{y}\bar{z} = w\bar{x}\bar{y}\bar{z} + \bar{w}\bar{x}\bar{y}\bar{z}$ steckt schon in $w\bar{x}\bar{y}$ und $\bar{w}\bar{x}\bar{z}$.

Beide Formen brauchen hier 3 Terme mit zusammen 8 Literalen. Welche günstiger ist, hängt von der Funktion ab: Bei Segment d im nächsten Abschnitt gewinnt die KNF.

Don't-Care-Einträge in der Wahrheitstafel

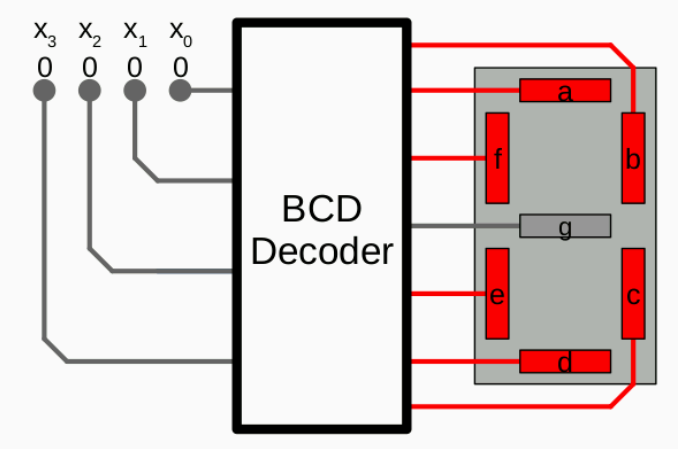
Am Anfang stand das Kreuz aus dem Datenblatt: Ein Wert, der keine Rolle spielt. Am Ausgang einer Funktion ist das ein Geschenk, denn Sie dürfen für diese Fälle wählen, was die Schaltung kleiner macht.

In einigen Fällen bildet die Wahrheitstafel Kombinationen der Eingangswerte ab, die für die Ausgabe gar nicht relevant sind. In diesem Fall spricht man von sogenannten `don't care` Ausgaben. Letztendlich ist uns das Funktionsergebnis für diese Fälle egal. Aus den `don't care` Fällen können entsprechend weitere Minimierungen hergeleitet werden.

Ein sehr anschauliches Anwendungsbeispiel dafür sind Sieben-Segment-Anzeigen, die aus 7 + 1 (Punkt) LEDs bestehen und insbesondere zur Darstellung von Zahlenwerten genutzt werden (Datenblatt einer Anzeige).

Acht Eingänge pro Element würden die Zahl der verfügbaren Pins eines Controllers recht schnell aufbrauchen und sind auch nicht nötig. Wir wollen ja nur die Zahlen von 0 bis 9 darstellen. Diese 10 Zustände können wir mit 4 Leitungen abbilden (BCD-Code: jede Ziffer als vierstellige Binärzahl). Ein Decoder-Baustein erzeugt daraus die Signale für die Segmente, und genau dessen Funktionen minimieren wir hier.

Von den 16 möglichen Kombinationen dieser 4 Leitungen werden also 6 nie vorkommen. Sie sind für unsere Ausgabe irrelevant.



Das Segment d ist das untere waagerechte Segment. Es leuchtet bei allen Ziffern außer 1, 4 und 7.

Betrachten wir die Wahrheitstafel für das „d“ Element.

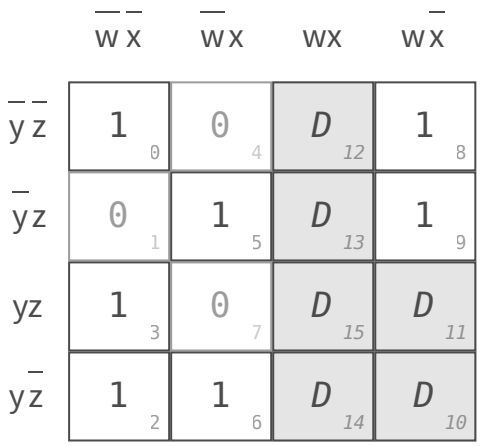
w	x	y	z	d
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	D
1	0	1	1	D
1	1	0	0	D
1	1	0	1	D
1	1	1	0	D
1	1	1	1	D

Für jedes D dürfen Sie selbst entscheiden: Hilft es, eine Schleife zu vergrößern, nehmen Sie es hinein. Sonst lassen Sie es liegen. Ein D muss nie überdeckt werden.

	$\bar{w}\bar{x}$	$\bar{w}x$	wx	$w\bar{x}$
$\bar{y}\bar{z}$	1	0	D	1
$\bar{y}z$	0	1	D	1
yz	1	0	D	D
$y\bar{z}$	1	1	D	D

Welche Gleichung für d lesen Sie daraus ab? Bilden Sie die Schleifen selbst:

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)



Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 7 Einsen.

Mit den Don't-Cares reicht zum Beispiel w als Term, ganz ohne weitere Variablen. Vergleichen Sie im Diagramm: Löschen Sie die D-Einträge unter Felder eintragen, wird das Ergebnis deutlich länger. Und schalten Sie auf die KNF um: Hier lohnt sie sich, denn sie braucht nur drei Terme.

$d = w + y\bar{z} + \bar{x}y + x\bar{y}z + \bar{x}\bar{z}$ (5 Terme, 10 Literale)

Ohne Don't-Cares bräuchte man ebenfalls 5 Terme, aber 16 Literale.

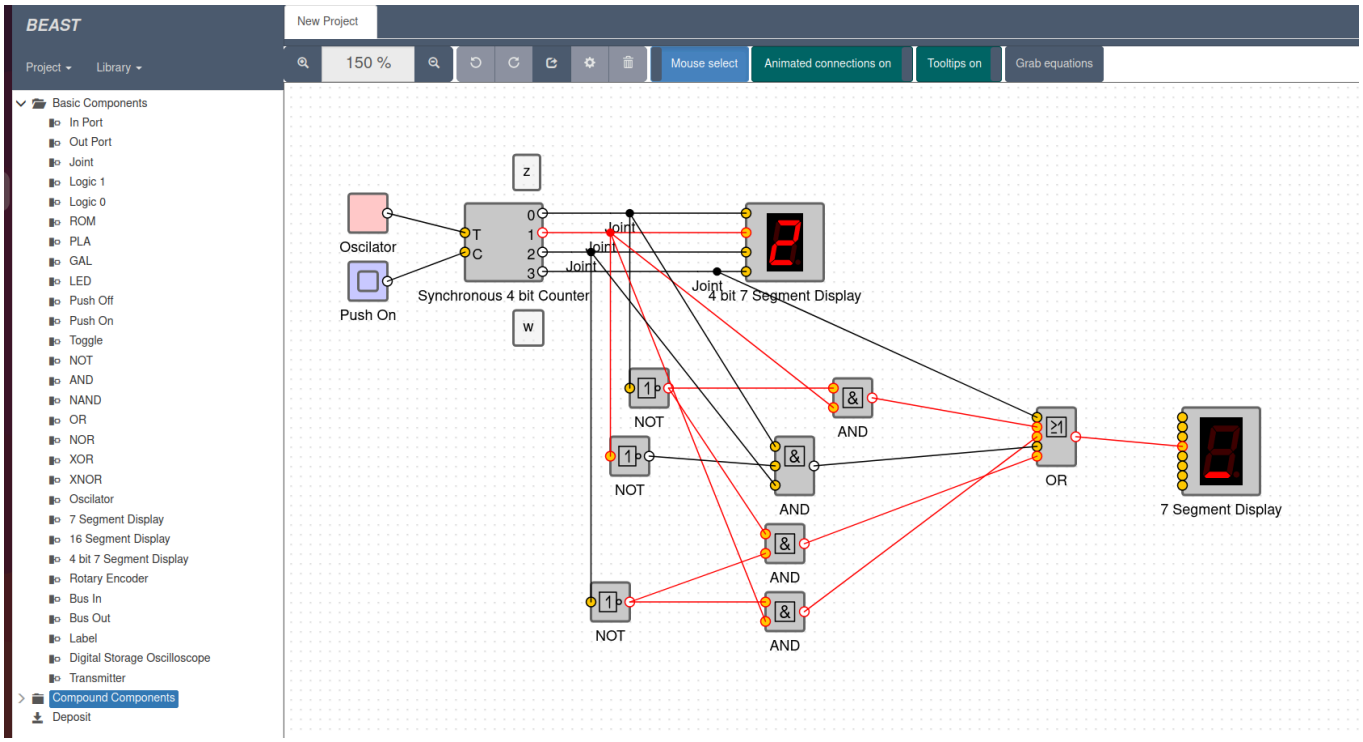
Die KNF lohnt sich hier sogar: $d = (\bar{x} + y + z)(w + x + y + \bar{z})(\bar{x} + \bar{y} + \bar{z})$ hat nur 3 Terme bei ebenfalls 10 Literalen. Prüfen Sie das oben mit „KNF (Nullen)“. Dort gilt ein D als 0, wenn es hilft.

Was gilt für einen D-Eintrag im Diagramm?

- ◯ Er muss von einer Schleife überdeckt werden.
- ◯ Er darf in eine Schleife, muss aber nicht.
- ◯ Er darf nur in Schleifen mit mindestens vier Feldern liegen.
- ◯ Er zählt immer als 0.

So sieht Segment d als Schaltung aus: Aus jedem Produktterm wird ein UND-Gatter, ein ODER fasst alle zusammen. Diesen Aufbau machen Sie in der Hausaufgabe selbst.

Die Gleichung als Schaltung: Jeder Produktterm wird zu einem UND-Gatter, das ODER fasst sie zusammen, der Term w geht direkt hinein. Im Bild erzeugt ein Zähler die Eingaben, die linke Anzeige zeigt die Ziffer und die rechte nur Segment d. In Aufgabe 5 bauen Sie das selbst auf.



Minimierung mit Software

Von Hand endet das Verfahren bei vier Variablen. Für mehr Variablen brauchen wir einen Algorithmus, und den schauen wir uns in der nächsten Vorlesung an. Ein Programm liefert die Lösung schon heute in wenigen Zeilen.

Für mehr als vier Variablen brauchen wir ein Verfahren, das ohne das Bild auskommt und sich automatisieren lässt. Das Verfahren von Quine-McCluskey folgt in der nächsten Vorlesung. Eine fertige Umsetzung einer solchen Minimierung bietet das Python-Paket SymPy:

Minimize.py

```
1  from sympy.logic import SOPform
2  from sympy import symbols
3  x3, x2, x1, x0 = symbols('x3 x2 x1 x0')
4
5  minterms = [[0, 1, 1, 0],   # jede Zeile der Liste: eine Zeile der
     Wahrheitstafel mit f = 1
6  [0, 1, 1, 1],
7  [1, 0, 0, 0],
8  [1, 0, 0, 1],
9  [1, 0, 1, 0],
10 [1, 0, 1, 1],
11 [1, 1, 0, 0],
12 [1, 1, 0, 1],
13 [1, 1, 1, 0]]
14 result = SOPform([x3, x2, x1, x0], minterms)
15 print("Minimized result:")
16 print(result)
```

```
Minimized result:
(x3 & ~x0) | (x3 & ~x1) | (x3 & ~x2) | (x1 & x2 & ~x3)
```

Die Dokumentation zum Sympy Paket von Python finden Sie unter https://docs.sympy.org/latest/modules/logic.html `SOPform` fasst dazu Terme paarweise zusammen, ähnlich wie das Verfahren von Quine-McCluskey aus der nächsten Vorlesung. Die Ausgabe nutzt die Python-Schreibweise: `&` für UND, `|` für ODER, `~` für NICHT.

Und im ATmega328P?

Hier zählen keine Gatter, sondern Bytes Programmspeicher. Lohnt sich das Minimieren trotzdem? Dieselbe Funktion für Segment d steht unten dreimal in C. Nachgemessen wurde mit dem Compiler avr-gcc, der auch hinter der Arduino-Umgebung steckt.

Auf dem ATmega328P wird eine Schaltfunktion zu einer C-Bedingung. Hier steht Segment `d` einmal als KDNF, einmal in der minimierten Form aus dem KV-Diagramm und einmal als Tabelle in einer Konstanten: Bit n ist der Wert von `d` für die Ziffer n, `>>` schiebt es nach ganz rechts, `& 1` schneidet es heraus. Alle drei bekommen die Ziffer als Zahl `n`.

Die Wahrheitstafel für Segment `d` kennen Sie aus dem Abschnitt zu den Don't-Cares. Die Spalte „Bit“ zeigt, was davon in der Konstanten von `d_tab` steht: Bit n gehört zu Zeile n, und aus jedem D wird eine 0. Von unten nach oben gelesen ergibt die Spalte die Konstante `0b0000001101101101`.

n	w	x	y	z	d	**Bit**
0	0	0	0	0	1	1
1	0	0	0	1	0	0
2	0	0	1	0	1	1
3	0	0	1	1	1	1
4	0	1	0	0	0	0
5	0	1	0	1	1	1
6	0	1	1	0	1	1
7	0	1	1	1	0	0
8	1	0	0	0	1	1
9	1	0	0	1	1	1
10	1	0	1	0	D	0
11	1	0	1	1	D	0
12	1	1	0	0	D	0
13	1	1	0	1	D	0
14	1	1	1	0	D	0
15	1	1	1	1	D	0

Segment_d.c

```
bool d_kdnf(uint8_t n) {
  bool w = n & 8, x = n & 4, y = n & 2, z = n & 1;
  return (!w && !x && !y && !z) || (!w && !x && y && !z) || (!w && !x && y
      || (!w && x && !y && z)   || (!w && x && y && !z)
      || (w && !x && !y && !z)  || (w && !x && !y && z);
}

bool d_min(uint8_t n) {
  bool w = n & 8, x = n & 4, y = n & 2, z = n & 1;
  return w || (y && !z) || (!x && y) || (x && !y && z) || (!x && !z);
}

bool d_tab(uint8_t n) {
  // Spalte d der Wahrheitstafel: Bit n = Zeile n
  return (0b0000001101101101 >> n) & 1;
}
```

Vier Bit Eingabe, sechzehn Bit Tabelle: Der kleinste Ganzzahltyp ist ein Byte, `uint8_t`. Die Ziffer belegt davon nur die unteren vier Bits: `n & 8` liest Bit 3 als w, `n & 4` Bit 2 als x, `n & 2` Bit 1 als y und `n & 1` Bit 0 als z. Die oberen vier Bits werten `d_kdnf` und `d_min` gar nicht aus, für sie sind 2, 18 und 130 dieselbe Eingabe. Die Konstante in `d_tab` ist etwas anderes: Sie ist die Spalte `d` der Wahrheitstafel als Zahl, ein Bit je Zeile. Vier Eingangsbits unterscheiden 16 Zeilen, also hat sie 16 Bit. Für die Don't-Cares 10 bis 15 stehen dort Nullen, kürzer geschrieben ist das `0b1101101101`. Weil die Konstante nicht in ein Byte passt, ist sie ein `int` mit 16 Bit. Ein Schieben um 16 oder mehr Stellen ist in C undefiniert, `d_tab` verträgt also nur Eingaben bis 15.

Variante	**Programmspeicher (`avr-gcc -Os`, optimiert auf Größe)**
KDNF	60 Byte
minimiert mit Don't-Cares	48 Byte
Tabelle	22 Byte

- **Minimierung spart auch hier**, wenn auch weniger als in Hardware, weil der Compiler den Ausdruck selbst vereinfacht. Die Don't-Cares kennt er allerdings nicht. Fest verdrahtet zählt jedes Gatter weiter, auch in der Logik im Inneren des ATmega328P selbst.
- **Software hat mehr Möglichkeiten:** Die Tabelle ist die kleinste Variante und braucht keine Boolesche Formel. In der Praxis legt man für jede Ziffer ein Byte mit allen Segmenten ab. Das kostet Zeit: Der AVR schiebt nur um eine Stelle je Befehl, `d_tab` braucht deshalb eine Schleife, deren Dauer von `n` abhängt.
- **Rechnet der Controller die Segmente selbst aus, belegt er sieben Ausgänge.** Nur mit einem externen Decoder kommt er mit vier Leitungen aus.
- **Don't Care ist eine Annahme, die jemand garantieren muss.** Für die Eingaben 10 bis 15 liefert `d_min` den Wert 1, die beiden anderen 0. Prüft das Programm die Eingabe nicht, zeigt die Anzeige Unsinn.

Was Sie heute mitnehmen

Diese Punkte fassen die Vorlesung zusammen. Die Fragen darunter gehören zu den Lernzielen vom Anfang: Wenn Sie sie beantworten und ein Karnaugh-Veitch-Diagramm selbst ausfüllen und auswerten können, sind Sie gut auf das Verfahren von Quine-McCluskey in der nächsten Woche vorbereitet.

- **Normalformen:** Eine DNF ist eine Disjunktion von Produkttermen, eine KNF eine Konjunktion von Summentermen. In der **KDNF** enthält jeder Term als **Minterm** alle Variablen, in der **KKNF** jeder Term als **Maxterm** – beide sind bis auf die Reihenfolge eindeutig.
- **Aus der Wahrheitstafel:** Jede Zeile mit $f = 1$ liefert einen Minterm der KDNF, jede Zeile mit $f = 0$ einen Maxterm der KKNF. Die KDNF lohnt sich, wenn nur wenige Einsen vorkommen, die KKNF bei wenigen Nullen.
- **Das Karnaugh-Veitch-Diagramm** ordnet die Zeilen der Wahrheitstafel so an, dass sich benachbarte Felder – auch über den Rand hinweg – nur in einer Variablen unterscheiden (Gray-Code). Zusammengefasste Felder fallen nach $x + \bar{x} = 1$ zu einem kürzeren Term zusammen.
- **Schleifen** sind Rechtecke aus 2^k Feldern; gesucht sind möglichst wenige, möglichst große Schleifen, die alle Einsen überdecken. Fasst man stattdessen die Nullen zusammen, erhält man die KNF.
- **Don't-Care-Einträge** markieren Eingangskombinationen, deren Ausgabe egal ist – etwa die sechs ungenutzten Codes bei der Ansteuerung einer Siebensegmentanzeige. Sie dürfen in Schleifen einbezogen werden, wo sie diese vergrößern.

Prüfen Sie sich:

- Welcher Unterschied besteht zwischen der DNF und der KDNF?
- Welcher Unterschied besteht zwischen der DNF und der KNF?
- Wie stellen Sie aus einer Wahrheitstafel die KDNF und die KKNF auf, und wann ist welche Form günstiger?
- Welchem Grundkonzept folgt die Anordnung der Felder im Karnaugh-Veitch-Diagramm?
- Welche Bereiche dürfen im Diagramm zu einer Schleife zusammengefasst werden?
- Was bedeutet „minimal“, und was zählt als Literal?
- Worin unterscheidet sich das Vorgehen im Karnaugh-Veitch-Diagramm für Minterme und für Maxterme?
- Was sind Don't-Care-Einträge, und wie nutzen Sie diese bei der Minimierung?

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; die Begriffe dieser Woche sind seit Beginn der Vorlesung in der Karte freigeschaltet, und Ihre Vorschläge ergänzen sie.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe ...

Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?

Texteingabe ...

Hausaufgaben

Bis zum Beginn der Übungen Ende November ersetzt die Hausaufgabe die Übung. Planen Sie 60–75 Minuten ein. Teil A prüfen Sie direkt hier, Teil B besprechen wir zu Beginn der nächsten Vorlesung.

Teil A – mit Selbstkontrolle

Aufgabe 1: Lösen Sie das Minimierungsproblem der Einstiegsaufgabe mit dem Karnaugh-Veitch-Diagramm. Die Einsen von f sind schon eingetragen.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\overline{x_4}\,\overline{x_3}$	$\overline{x_4}\,x3$	$x4\,x3$	$x4\,\overline{x_3}$
$\overline{x_2}\,\overline{x_1}$	0 0	1 4	1 12	1 8
$\overline{x_2}\,x1$	0 1	0 5	1 13	1 9
$x2\,x1$	0 3	0 7	0 15	0 11
$x2\,\overline{x_1}$	0 2	1 6	1 14	0 10

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

$f = ?$

Noch nicht überdeckt: 7 Einsen.

Welche Zeile der Wahrheitstafel wird von beiden Schleifen überdeckt?

Aufgabe 2: Minimieren Sie Segment a, das obere waagerechte Segment. Es leuchtet bei den Ziffern 0, 2, 3, 5, 6, 7, 8 und 9. Die Eingaben 10 bis 15 kommen nicht vor. Die Variablen sind wie oben die BCD-Bits w, x, y, z. Die kleine Zahl im Feld ist die Zeile der Wahrheitstafel, hier also die Ziffer. Tragen Sie die Felder 10 bis 15 als D ein.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\overline{w}\,\overline{x}$	$\overline{w}\,x$	$w\,x$	$w\,\overline{x}$
$\overline{y}\,\overline{z}$	0 0	0 4	0 12	0 8
$\overline{y}\,z$	0 1	0 5	0 13	0 9
$y\,z$	0 3	0 7	0 15	0 11
$y\,\overline{z}$	0 2	0 6	0 14	0 10

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Klick wechselt 0 → 1 → D.

$f = 0$

Welcher Ausdruck ist minimal?

- ◯ $w + y + xz + \bar{x}\bar{z}$
- ◯ $\bar{w}y + \bar{w}xz + w\bar{x}\bar{y} + \bar{w}\bar{x}\bar{z}$
- ◯ $w + x + y + z$
- ◯ $w + xz$

Aufgabe 3: Bestimmen Sie die minimale KNF der Funktion Y aus „Und jetzt Sie!“ im Abschnitt Beispiele ($Y = 1$ für die Zeilen 0, 1, 2 und 4). Schalten Sie das Diagramm dazu auf „KNF (Nullen)“.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
$\bar{C}$	1 (0)	1 (2)	0 (6)	1 (4)
C	1 (1)	0 (3)	0 (7)	0 (5)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 4 Einsen.

Welcher Ausdruck ist die minimale KNF?

- ◯ $(A + B)(A + C)(B + C)$
- ◯ $(\bar{A} + \bar{B})(\bar{A} + \bar{C})(\bar{B} + \bar{C})$
- ◯ $\bar{A}\bar{B} + \bar{A}\bar{C} + \bar{B}\bar{C}$
- ◯ $A + B + C$

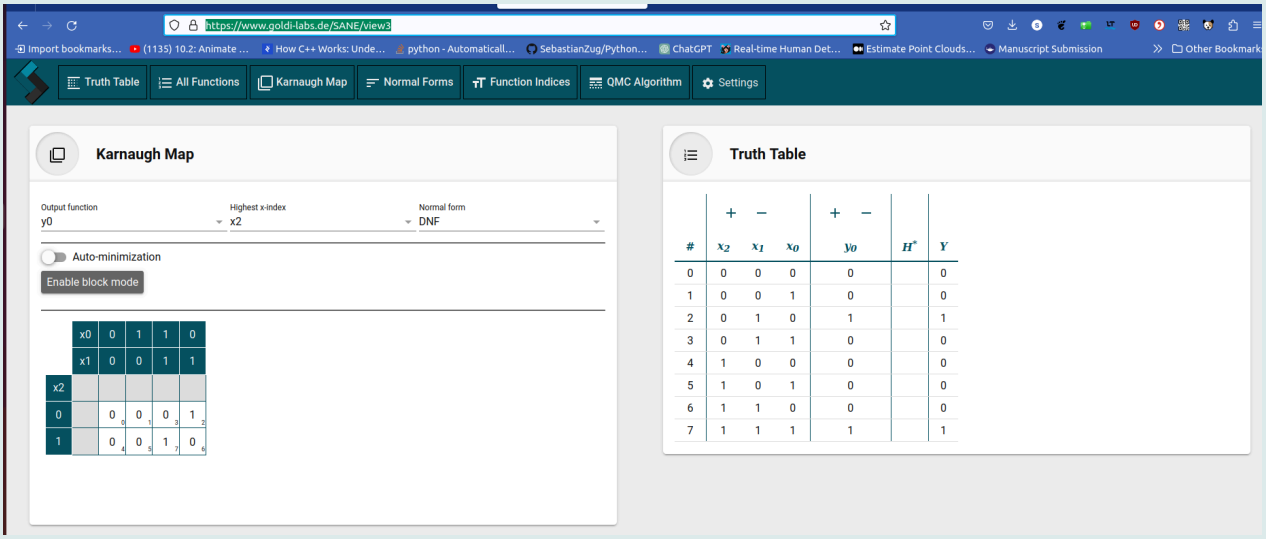
Teil B – Besprechung in der nächsten Vorlesung

Aufgabe 4: Ändern Sie `Minimize.py` so, dass es Segment `d` minimiert: Tragen Sie die Einsen in `minterms` ein und übergeben Sie die Eingaben 10 bis 15 als dritten Parameter von `SOPform`. Vergleichen Sie das Ergebnis mit dem Abschnitt zu den Don't-Cares. Weicht es ab, warum darf es das? Lassen Sie danach den dritten Parameter weg. Was ändert sich, und für welche Eingaben liefert die Funktion nun andere Werte?

Aufgabe 5: Bauen Sie Segment `d` selbst auf, im Simulator BEAST, der ohne Installation im Browser läuft. Ziehen Sie aus „Basic Components“ vier `Toggle` für w, x, y, z, ein `4 bit 7 Segment Display` für die Ziffer, die Gatter `NOT`, `AND` und `OR` sowie eine `LED` für `d` auf die Arbeitsfläche und verdrahten

Sie sie nach der Gleichung aus dem Abschnitt zu den Don't-Cares. Prüfen Sie die Ziffern 0 bis 9: Leuchtet die LED genau dann, wenn das untere Segment der Ziffer leuchtet? Was zeigen LED und Anzeige bei den Eingaben 10 bis 15?

Zum weiteren Training können Sie die Werkzeuge der TU Ilmenau verwenden: https://goldi-labs.net/SANE/



Nächste Vorlesung

Am **Mittwoch, 11.11.**, geht es um Schaltnetze: Das Verfahren von Quine-McCluskey minimiert auch Funktionen mit mehr als vier Variablen.